

题目链接

本题是2011年ICPC亚洲区域赛大连赛区的B题

[大连2011的完整提单](#)

题意

给你w*h个格子(1≤w,h≤10000000) 然后给你n(0≤n≤50000)个矩形，表示这些矩形所在的格子被占据。然后需要在同一直线上放一个长为m的新机器（不可经过被占据的格子）。问有多少种放的方法。

分析

先考虑横向摆放新机器，也就是左端点可以摆放在哪些位置：按题意每个矩形任意行左侧的m-1个格子是不能摆放的并且每个矩形占据的格子也不能摆放，同时有任意行边界左侧的m-1个格子也不能摆放。

因此，不能摆放的点形成若干个矩形区域（这些矩形可能重叠），要求并。

这是线段树的经典应用：[线段树求矩形面积并](#)。

具体来说有一个扫描线的概念对数据做离散化处理，再构建线段树求面积并（其实就处理区间的覆盖与撤销）。

若m>1，还需要计算一次纵向摆放的情况。

由于离散化数据后，每条扫描线的左右边界L/R对应左闭右开区间，即[L,R)，线段树的内部逻辑需要调整一些细节，适配这种情况。

AC代码

```
#include <iostream>
#include <algorithm>
using namespace std;

#define N 100100
struct node {
    int l, r, v; long long h;
    node (int l=0, int r=0, int v=0, long long h=0): l(l), r(r), h(h), v(v) {}
    bool operator< (const node& rhs) const {
        return h < rhs.h;
    }
} s[N];
int a[1<<18], c[1<<18], x[N], x1[N>>1], x2[N>>1], y1[N>>1], y2[N>>1], m, n, t; long long ans, w, h;

void build(int o, int l, int r) {
    a[o] = c[o] = 0;
    if (l == r) return;
    int m = (l+r)>>1, lc = o<<1, rc = lc|1;
    build(lc, l, m); build(rc, m+1, r);
}

void update(int o, int l, int r) {
    if (a[o]) c[o] = x[r+1] - x[l];
    else if (l == r) c[o] = 0;
    else c[o] = c[o<<1] + c[o<<1|1];
}

void add(int o, int l, int r, int y, int z, int v) {
    if (x[l] >= y && x[r+1] <= z) a[o] += v;
    else {
        int m = (l+r)>>1, lc = o<<1, rc = lc|1;
        if (y < x[m+1]) add(lc, l, m, y, z, v);
        if (z > x[m+1]) add(rc, m+1, r, y, z, v);
    }
    update(o, l, r);
}

void calc() {
    sort(s, s+t); sort(x, x+t);
    int k = unique(x, x+t) - x - 2;
    build(1, 0, k);
    for (int i=0, j=t-1; i<j; ++i) {
        add(1, 0, k, s[i].l, s[i].r, s[i].v);
        ans -= c[1] * (s[i+1].h - s[i].h);
    }
}

void solve() {
    ans = 0; t = n << 1;
    for (int i=0; i<n; ++i) cin >> x1[i] >> y1[i] >> x2[i] >> y2[i], --x1[i], --y1[i];
    if (w >= m) {
        ans = (w-m+1)*h;
        if (t) {
            for (int i=0, y = w-m+1; i<n; ++i) {
                x[i] = max(x1[i]-m+1, 0); x[i+n] = min(x2[i], y);
                s[i] = node(x[i], x[i+n], 1, y1[i]); s[i+n] = node(x[i], x[i+n], -1, y2[i]);
            }
            calc();
        }
    }
    if (h >= m && m > 1) {
        ans += (h-m+1)*w;
        if (t) {
            for (int i=0, y = h-m+1; i<n; ++i) {
                x[i] = max(y1[i]-m+1, 0); x[i+n] = min(y2[i], y);
                s[i] = node(x[i], x[i+n], 1, x1[i]); s[i+n] = node(x[i], x[i+n], -1, x2[i]);
            }
            calc();
        }
    }
    cout << ans << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    while (cin >> w >> h >> n >> m) solve();
    return 0;
}
```